QubitVisualization

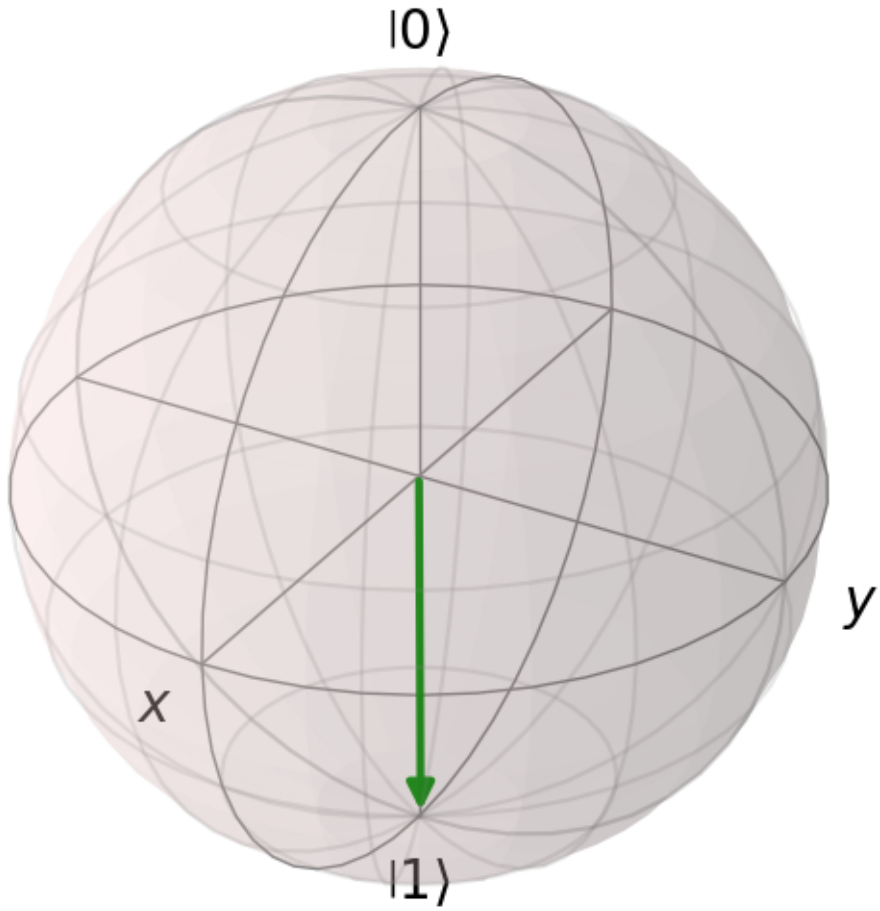
October 30, 2025

[1]:
```
import sys
import matplotlib.pyplot as plt
import qutip
import cudaq
import numpy as np
```

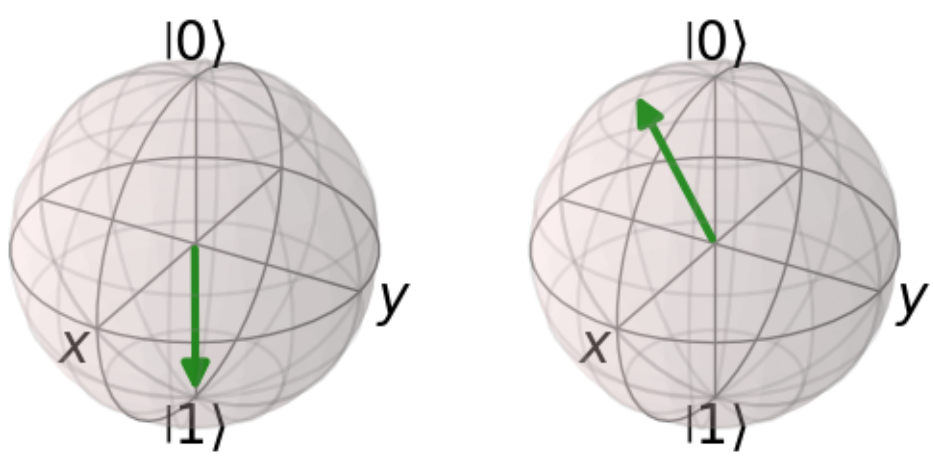
[2]:
```
@cudaq.kernel
def kernel(angles: np.ndarray):
    qubit = cudaq.qubit()
    rz(angles[0], qubit)
    rx(angles[1], qubit)
    rz(angles[2], qubit)
```

[3]:
```
rng = np.random.default_rng(seed=11)
blochSphereList = []
for _ in range(4):
    angleList = rng.random(3) * 2 * np.pi
    sph = cudaq.add_to_bloch_sphere(cudaq.get_state(kernel, angleList))
    blochSphereList.append(sph)
```

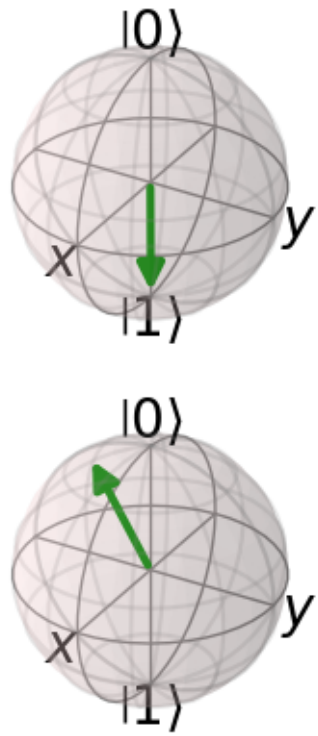
[4]:
```
cudaq.show(blochSphereList[0])
```



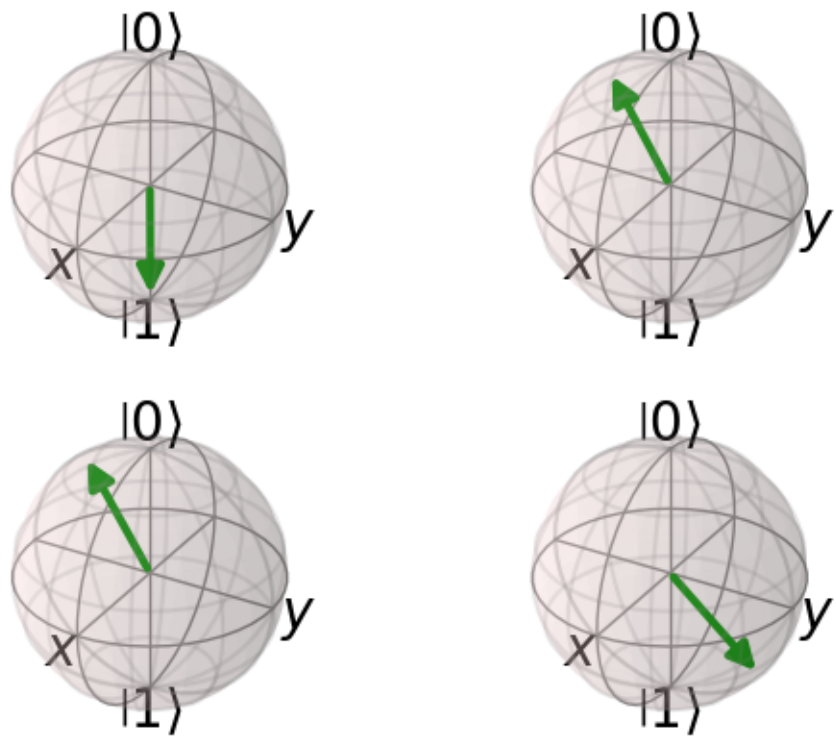
[5]:
```
cudaq.show(blochSphereList[:2], nrows=1, ncols=2)
```



[6]:
```
cudaq.show(blochSphereList[:2], nrows=2, ncols=1)
```

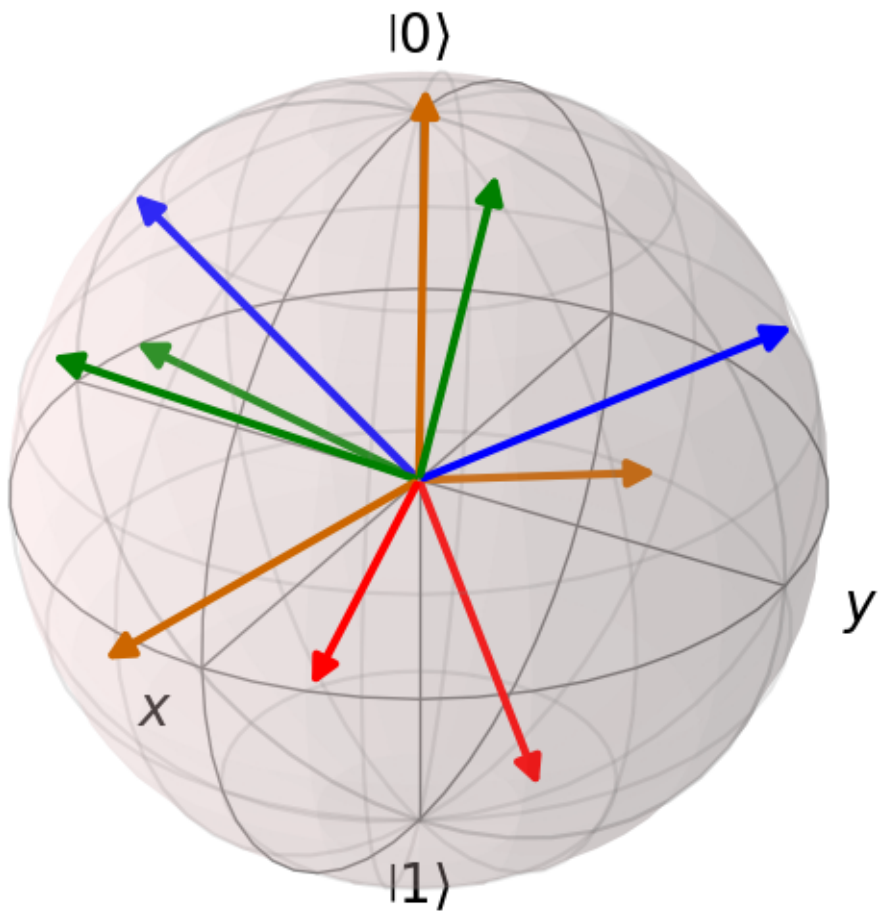


[7]:
```
cudaq.show(blochSphereList[:], nrows=2, ncols=2)
```



[8]:
```
rng = np.random.default_rng(seed=47)
blochSphere = qutip.Bloch()
for _ in range(10):
    angleList = rng.random(3) * 2 * np.pi
    sph = cudaq.add_to_bloch_sphere(cudaq.get_state(kernel, angleList),␣
 ↪blochSphere)
```

[9]:
```
blochSphere.show()
```



[10]:
```
@cudaq.kernel
def kernel_to_draw():
    q = cudaq.qvector(4)
    h(q)
    x.ctrl(q[0], q[1])
    y.ctrl([q[0], q[1]], q[2])
    z(q[2])

    swap(q[0], q[1])
    swap(q[0], q[3])
    swap(q[1], q[2])

    r1(3.14159, q[0])
    tdg(q[1])
    s(q[2])
```

[11]:
```
print(cudaq.draw(kernel_to_draw))
```

```
q0 :  h            r1(3.142)

q1 :  h   x                tdg

q2 :  h      y   z            s

q3 :  h
```

[12]:
```
print(cudaq.draw('latex', kernel_to_draw))
```

```
\documentclass{minimal}
\usepackage{quantikz}
\begin{document}
\begin{quantikz}
  \lstick{$q_0$} & \gate{H} & \ctrl{1} & \ctrl{2} & \swap{1} & \swap{3} &
\gate{R_1(3.142)} & \qw & \qw \\
  \lstick{$q_1$} & \gate{H} & \gate{X} & \ctrl{1} & \targX{} & \qw & \swap{1} &
\gate{T^\dag} & \qw \\
  \lstick{$q_2$} & \gate{H} & \qw & \gate{Y} & \gate{Z} & \qw & \targX{} &
\gate{S} & \qw \\
  \lstick{$q_3$} & \gate{H} & \qw & \qw & \qw & \targX{} & \qw & \qw & \qw \\
\end{quantikz}
\end{document}
```

[ ]:
```
```